Rapport de Défis 2:

Dans tout ce qui suit, j'ai répondu uniquement aux parties où il y a des questions, pour la partie RL2, étant donné qu'il n'y a des questions sur lesquelles on doit répondre à chaque fois il est demandé de tester et d'implémenter dans les fichiers de code, donc j'ai fait une petite analyse sur ce qui est demandé.

RL1 :

Question:Commenter les résultats obtenus: les valeurs et les politiques semblent-elles raisonnables ?

L'algorithme d'itération de valeur a été testé sur 10 épisodes. Voici les retours observés pour chaque épisode :

1. **Episode 1** : Return = 0.531441
2. **Episode 2** : Return = 0.590490
3. **Episode 3** : Return = 0.430467
4. **Episode 4** : Return = 0.590490
5. **Episode 5** : Return = 0.387420
6. **Episode 6** : Return = 0.590490
7. **Episode 7** : Return = 0.590490
8. **Episode 8** : Return = 0.478297
9. **Episode 9** : Return = 0.478297
10. **Episode 10** : Return = 0.590490

La moyenne des retours obtenus pour ces 10 épisodes est de **0.5258**. Cela indique une convergence relativement stable des retours, avec des valeurs qui oscillent autour de 0.5.

La majorité des retours se stabilisent autour de 0.590490, ce qui suggère que l'agent a appris une politique relativement optimale au fur et à mesure des épisodes.

Il n'y a pas de comportements anormaux évidents dans les retours, et les fluctuations peuvent être interprétées comme un aspect normal de l'exploration et de l'amélioration de la politique de l'agent.

Question:Changez un seul de ces paramètres de façon à obtenir une politique capable de traverser le pont et expliquez votre raisonnement.

Dans cette situation, un ajustement du **facteur de dépréciation** (discount rate, γ) pourrait être efficace.

En augmentant γ, l'agent sera plus motivé à atteindre la récompense élevée à droite, car les valeurs des états futurs auront plus d'importance. Même si l'agent peut tomber à l'eau en raison du bruit, il pourrait considérer que le bénéfice à long terme (atteindre la valeur de 10 à droite) compense le risque de tomber à l'eau.

Question : Rappelez le rôle de chacun des paramètres : facteur d'appréciation (discount), bruit (noise) et récompense (livingReward).

Facteur de dépréciation (discount) : Ce paramètre contrôle l'importance des récompenses futures par rapport aux récompenses immédiates. Un γ élevé (par exemple, 0.9) incite l'agent à privilégier les récompenses futures, tandis qu'un γ plus faible incite à privilégier les récompenses immédiates.

Bruit (noise) : Ce paramètre représente l'incertitude dans les actions de l'agent. Si le bruit est élevé (par exemple, 0.2), l'agent risque de ne pas exécuter exactement l'action qu'il souhaite, ce qui rend la navigation plus chaotique.

Récompense de vie (living Reward) : Ce paramètre définit la récompense ou la pénalité pour chaque état traversé.

Question:Trouvez les valeurs de ces param`etres afin que le processus de d´ecision de Markov produise des politiques optimales avec les effets suivants, si c'est possible

a) Préférer la sortie proche (+1) en risquant les états négatifs (-10).

Solution : Augmenter la valeur de **livingReward** pour encourager l'agent à aller vite vers la récompense proche, même si cela implique de risquer des états négatifs.

python gridworld.py -a value -i 100 -g DiscountGrid --discount 0.9 --noise 0.2 --livingReward -0.1

(b) Préférer la sortie proche (+1) en évitant les états négatifs (-10)

Solution : L'agent doit être découragé de passer par les états négatifs. Une valeur légèrement négative pour **livingReward** peut encourager l'agent à éviter ces états sans le forcer à prendre trop de risques.

python gridworld.py -a value -i 100 -g DiscountGrid --discount 0.9 --noise 0.2 --livingReward -0.01

(c) Préférer la sortie lointaine (+10) en risquant les états négatifs (-10)

Solution : Diminuer la valeur de **livingReward** pour augmenter la valeur des récompenses futures, incitant l'agent à prendre les risques pour la récompense élevée.

python gridworld.py -a value -i 100 -g DiscountGrid --discount 0.9 --noise 0.2 --livingReward -0.05

(d) Éviter les deux sorties et les états négatifs (de façon à ce qu'un épisode ne se termine jamais)

Solution : Un **livingReward** négatif élevé et une valeur de `discount` faible devraient encourager l'agent à rester dans la grille sans se diriger vers les états terminaux. Cela incite l'agent à naviguer indéfiniment.

python gridworld.py -a value -i 100 -g DiscountGrid --discount 0.2 --noise 0.2 --livingReward -0.5

RL2:

Phase d'apprentissage (Q-learning standard)

Au début de l'implémentation, j'ai utilisé un agent PacmanQAgent, qui est basé sur un Q-learning standard avec des paramètres d'apprentissage spécifiques ($\epsilon = 0.05$, $\alpha = 0.2$, $\gamma = 0.8$). Durant la phase d'apprentissage, l'agent a effectué plusieurs milliers d'itérations pour explorer l'espace d'état et d'action. Pour chaque épisode, l'agent a appris à ajuster sa fonction de valeur Q pour chaque état-action afin de maximiser la récompense à long terme.

Résultats de l'apprentissage (smallGrid)

Pour tester l'agent sur une grille simplifiée (smallGrid), j'ai effectué 2000 épisodes d'apprentissage suivis de 2010 épisodes de test. Au cours de l'apprentissage, l'agent a progressé lentement, mais à la fin, il a réussi à obtenir des récompenses positives sur une centaine d'épisodes. Le résultat final a montré que l'agent était capable de gagner au moins 80% du temps après avoir appris une politique optimale. Pendant l'apprentissage, l'agent a parfois obtenu de mauvais résultats à cause de l'exploration ($\epsilon > 0$).

Phase de test et visualisation

J'ai visualisé les 10 premiers jeux d'entraînement pour observer le processus d'apprentissage. Cela m'a permis de voir l'impact de l'exploration sur les résultats obtenus en début d'apprentissage, avec des récompenses souvent négatives lorsque l'agent se retrouvait face à un fantôme ou se déplaçait de manière non optimale. Cependant, à mesure de l'entraînement, les résultats se sont améliorés et les récompenses sont devenues plus positives.

Entraînement sur des grilles plus complexes (mediumGrid et mediumClassic)

Ensuite, j'ai testé l'agent sur des grilles plus grandes (mediumGrid et mediumClassic). Sur mediumGrid, l'agent a rencontré plus de difficultés en raison de l'augmentation de la taille de l'espace d'état, ce qui a rallongé la phase d'apprentissage. Les récompenses restaient négatives pendant un certain temps, mais l'agent a progressivement appris à éviter les fantômes et à collecter les pac-dots.

Implémentation du Q-learning approximatif

Dans la deuxième partie du projet, j'ai implémenté un agent ApproximateQAgent utilisant le Q-learning approximatif. J'ai utilisé des caractéristiques extraites de l'état et de l'action via la méthode IdentityExtractor, qui associe une seule caractéristique à chaque paire état-action. Ce modèle approximatif est plus efficace pour gérer des espaces d'état de grande taille.

Ensuite, j'ai utilisé un extracteur de caractéristiques plus complexe (SimpleExtractor) pour améliorer l'apprentissage sur des grilles plus grandes comme mediumGrid et

mediumClassic. Grâce à ce modèle, l'agent a pu apprendre plus rapidement et a montré une performance presque optimale après seulement une cinquantaine de jeux d'entraînement.

7. Conclusion

En conclusion, l'implémentation du Q-learning pour PacMan a montré de bons résultats, notamment dans les phases d'apprentissage et de test. Le passage au Q-learning approximatif a permis d'améliorer l'efficacité de l'agent dans des environnements plus complexes. Grâce à cette approche, l'agent a réussi à apprendre à naviguer dans des grilles de tailles variables tout en maximisant ses récompenses.